

UMCS

UNIwersytet Marii Curie-Skłodowskiej w Lublinie

Wydział Matematyki, Fizyki i Informatyki

Kierunek: Informatyka

Maksim Ryshko

nr albumu: 310030

Wykorzystanie metod klastrowych podczas analizy danych tekstowych

Using clustering methods in text data analysis

Praca licencjacka

napisana w Katedrze Neuroinformatyki i Inżynierii Biomedycznej

pod kierunkiem dr Anny Gajos-Balińskiej

Lublin rok 2024

Spis treści

Wstęp	1
1. Techniki analizy danych	3
1.1. Klasteryzacja	3
1.2. Przetwarzanie języka naturalnego	5
1.3. Wektoryzacja	6
1.4. Redukcja wymiarowości	7
2. Wykorzystane metody i technologie	9
2.1. Opis implementacji	9
2.2. Dane tekstowe	10
2.3. Przetwarzania tekstu	11
2.4. Wektoryzacja	14
2.5. Klasteryzacja	16
2.6. Wizualizacja	19
3. Funkcje aplikacji	21
3.1. Opis aplikacji	21
4. Używanie aplikacji	27
4.1. Przykład tworzenia workspace	27
4.2. Przykład ustawienia parametrów i metod	27
Podsumowanie	37

Wstęp

W dzisiejszym świecie ilość danych tekstowych rośnie z niesamowitą prędkością. Dane tekstowe stają się nieodłączną częścią wielu obszarów, w tym mediów społecznościowych, forów, blogów i innych platform, gdzie użytkownicy aktywnie dzielą się swoimi opiniami i informacjami. Jedną z najpopularniejszych platform jest YouTube, na której miliony użytkowników codziennie zostawiają komentarze pod filmami. Analiza tych danych tekstowych może dostarczyć cenne informacje do badań marketingowych, społecznych oraz do poprawy interakcji z odbiorcami.

W ramach niniejszej pracy opracowana będzie aplikacja GUI przeznaczona do analizy danych tekstowych. Dane tekstowe mogą pochodzić z dowolnych dokumentów tekstowych. Projekt skupia się na przetwarzaniu komentarzy z YouTube, co umożliwi użytkownikom łatwe i szybkie pobieranie oraz analizowanie tych danych. Po wybraniu dostępnych metod przetwarzania komentarzy, możliwa będzie ocena wyniku za pomocą dwuwymiarowego wykresu, na którym dane tekstowe zostaną wizualnie podzielone na grupy (klastry). Aby pomóc w jakościowej analizie danych, użytkownik będzie mógł dostosować metody przetwarzania języka naturalnego (NLP), wektoryzacji i klasteryzacji.

W pierwszym rozdziale zostaną omówione techniki analizy danych. W drugim rozdziale – wybrane metody i technologie do implementacji aplikacji. W trzecim rozdziale przedstawiony zostanie wygląd oraz funkcje aplikacji. W czwartym rozdziale przeprowadzony zostanie przykład użycia aplikacji.

Rozdział 1

Techniki analizy danych

W tym rozdziale omówiono techniki analizy danych, które są wykorzystywane w wielu dziedzinach nauki i przemysłu. Przedstawiono klasteryzację, przetwarzanie języka naturalnego, wektoryzację i metody redukcji wymiarowości.

1.1. Klasteryzacja

Klasteryzacja (ang. clustering) to wielowymiarowa procedura analizy statystycznej, mająca na celu zbieranie danych zawierających informacje (cechy) o próbie obiektów, a następnie porządkowanie tych obiektów w stosunkowo jednorodne grupy. Zakres zastosowania klasteryzacji jest niezwykle szeroki: używa się jej w archeologii, medycynie, psychologii, chemii, biologii, administracji państwowej, filologii, antropologii, marketingu, socjologii, geologii i innych dyscyplinach [2].

1.1.1. Cele klasteryzacji

Klasteryzacja jest najczęściej wykorzystywana do następujących celów:

1. **Zrozumienie danych poprzez identyfikację struktury klastrów.** Podział próby na grupy podobnych obiektów pozwala uprościć dalsze przetwarzanie danych i podejmowanie decyzji, stosując dla każdego klastra odpowiednią metodę analizy.
2. **Kompresja danych.** Jeśli pierwotna próba jest zbyt duża, można ją skompresować, zachowując tylko jednego najbardziej typowego przedstawiciela z każdego klastra. Zastępując wszystkie obiekty w każdym klastrze jednym najbardziej typowym przedstawicielem, można znacznie zmniejszyć objętość danych, zachowując jednocześnie główne cechy i strukturę pierwotnych danych.
3. **Wykrywanie nowości (odchyłeń lub anomalii).** Klasteryzacja pozwala na wyróżnienie nietypowych obiektów, które nie mogą być przypisane do żadnego z klastrów. Te obiekty mogą być interesujące jako potencjalne anomalie lub odchylenia, wymagające dodatkowych badań, analiz lub uwagi.

1.1.2. Metody klasteryzacji

Nie ma powszechnie przyjętej klasyfikacji metod klasteryzacji, jednak można wyróżnić kilka grup podejść:

1. **Algorytmy klasteryzacji oparte na grafach.** Klasa ta obejmuje prymitywne algorytmy oparte na budowaniu grafu podobieństwa między obiektami. Obecnie algorytmy klasteryzacji oparte na grafach nie są stosowane w praktyce ze względu na ich dużą złożoność obliczeniową i ograniczoną skalowalność przy dużych zbiorach danych.

2. **Algorytmy klasteryzacji probabilistycznej.** Algorytmy te przypisują każdemu obiektowi z próby treningowej prawdopodobieństwo przynależności do każdego z klastrów.
3. **Hierarchiczne algorytmy klasteryzacji.** Algorytmy te porządkują dane, tworząc hierarchię zagnieżdżonych klastrów.
4. **Algorytmy oparte na gęstości danych.** Te algorytmy obejmują różne techniki. Pierwszym z nich jest **K-średnich (K-means)**, który jest iteracyjnym algorytmem opartym na minimalizacji sumy kwadratowych odchyleń punktów klastrów od ich centrów. Następnie jest **propagacja podobieństwa (affinity propagation)**, algorytm, który rozprzestrzenia wiadomości o podobieństwie między parami obiektów w celu wyboru typowych przedstawicieli każdego klastra. Kolejną metodą jest **przesunięcie średniej (mean shift)**, wybierająca centroidy klastrów w obszarach o największej gęstości. **Klasteryzacja spektralna (spectral clustering)** to metoda wykorzystująca wartości własne macierzy odległości do zmniejszenia wymiarowości przed zastosowaniem innych metod klasteryzacji. W końcu **DBSCAN (Density-Based Spatial Clustering of Applications with Noise)**, jest algorytmem grupującym punkty w jeden klastery w obszarach o wysokiej gęstości i oznaczający odosobnione punkty jako szum.

1.1.3. Etapy klasteryzacji

Niezależnie od przedmiotu badań, zastosowanie analizy klastrów obejmuje określone etapy. Pierwszym z nich jest wybór próby do klasteryzacji. Zakłada się, że klasyfikować można tylko dane ilościowe, ponieważ wskaźniki liczbowe mogą być ilościowo oceniane. Następnie określa się **zestaw zmiennych**, na podstawie których obiekty w próbie będą oceniane, tj. przestrzeni cech. Cechy mogą być różnymi charakterystykami obiektów, które mają znaczenie dla badań. Kolejnym etapem jest **obliczanie wartości wybranej miary podobieństwa (lub różnicy) między obiektami**. Celem jest określenie, w jakim stopniu obiekty w próbie są do siebie podobne lub różnią się względem wybranych cech. Następnie stosuje się **metodę analizy klastrów** do stworzenia grup podobnych obiektów. Zakłada się wykorzystanie jednego z istniejących algorytmów klasteryzacji w taki sposób, aby obiekty w jednym klastrze były maksymalnie podobne do siebie. Obiekty z różnych klastrów powinny maksymalnie się różnić. Ostatnim etapem jest **sprawdzenie wiarygodności wyników klasteryzacji**, co może być osiągnięte za pomocą różnych metod, takich jak wizualizacja klastrów, analiza stabilności klastrów i ocena jakości podziału.

1.1.4. Zastosowanie

Zastosowanie metody analizy klastrów jest z powodzeniem wykorzystywane w wielu dziedzinach i dyscyplinach. Metoda ta jest również szeroko stosowana w informatyce. Pierwszym przykładem wykorzystania klasteryzacji w pracy z danymi tekstowymi jest **klasteryzacja wyników wyszukiwania**, która polega na grupowaniu dokumentów zwróconych w odpowiedzi na zapytanie użytkownika, aby podobne dokumenty pojawiały się razem [6]. Następnym przykładem jest **grupowanie użytkowników o podobnych zainteresowaniach**, co pozwala na lepszą personalizację treści, rekomendacje oraz targetowanie w reklamie [7]. Ostatnim przykładem jest **grupowanie tekstowych dokumentów**, takich jak artykuły prasowe, publikacje naukowe czy wiadomości w mediach społecznościowych, które polega na organizowaniu ich według tematyki, co ułatwia użytkownikom odnalezienie interesujących ich treści [6].

1.1.5. Ocena efektywności klasteryzacji

Problem oceny jakości w zadaniu klasteryzacji jest trudny do rozwiązania z co najmniej dwóch powodów. Po pierwsze, zgodnie z twierdzeniem o niemożności Kleinberga [1], nie istnieje optymalny algorytm klasteryzacji. Po drugie, wiele algorytmów klasteryzacji nie jest w stanie samodzielnie określić rzeczywistej liczby klastrów w danych; najczęściej liczba klastrów jest podawana na wejściu algorytmu i dobierana kilkoma uruchomieniami. Niemniej jednak, wyróżnia

się dwie grupy metod oceny jakości klasteryzacji.

Zewnętrzne metody porównują wynik klasteryzacji z apriorycznie znanym podziałem na klasy. Przykładem zewnętrznej metody oceny może być indeks Randa. Metoda ta ocenia, jak wiele spośród par elementów, które znajdowały się w jednej klasie oraz tych, które znajdowały się w różnych klasach, zachowało ten stan po zastosowaniu algorytmu klasteryzacji.

Wewnętrzne metody oceniają jakość klasteryzacji, korzystając jedynie z informacji zawartych w samych danych. Przykładem wewnętrznej metody jest metoda spójności klastra (Cluster Cohesion). Idea tej metody polega na tym, że im bliżej siebie znajdują się obiekty w klastrach, tym lepszy jest podział. Można również wyróżnić metodę separacji klastra (Cluster Separation), która ocenia jakość klasteryzacji na podstawie odległości między obiektami różnych klastrów.

1.2. Przetwarzanie języka naturalnego

Przetwarzanie języka naturalnego (ang. Natural Language Processing, NLP) to dziedzina badań, która łączy metody uczenia maszynowego i lingwistyki matematycznej w celu opracowania algorytmów do analizy, rozumienia i generowania tekstu w językach naturalnych [3].

1.2.1. Wstępne przetwarzanie tekstu

Wstępne przetwarzanie tekstu jest ważnym etapem NLP, który przekształca tekst w języku naturalnym w format wygodny do rozpoznawania przez algorytmy i do dalszej pracy. Przetwarzanie tekstu składa się z różnych kroków, które mogą się różnić w zależności od zadania i implementacji. Oto niektóre z najpopularniejszych podejść:

- konwersja wszystkich liter w tekście na małe lub wielkie litery,
- usuwanie cyfr (liczb) lub zamiana na tekstowy ekwiwalent (zwykle używane są wyrażenia regularne),
- usuwanie znaków białych (whitespaces),
- tokenizacja (zwykle realizowana na podstawie wyrażeń regularnych),
- usuwanie stop-words,
- stemming,
- lematyzacja,
- wektoryzacja.

W kolejnych podrozdziałach szczegółowo omówiono trzy ostatnie metody, ponieważ są one bardziej skomplikowane pod względem koncepcyjnie i implementacyjnie.

1.2.2. Stemming

Stemming to proces redukcji słowa do jego podstawowej formy, zwanej rdzeniem lub stemem. Liczne formy wyrazowe różniące się sufiksami, przedrostkami czy końcówkami utrudniają tworzenie słowników i dalsze przetwarzanie. Stemming sprowadza słowo do jego podstawowej formy. Istota podejścia polega na znalezieniu rdzenia słowa. W tym celu z końca i początku słowa kolejno odcinane są jego części zgodnie z ustalonymi regułami. Wadą podejścia jest możliwa utrata kontekstu, ponieważ możemy stracić informację o części mowy.

Przykładem przetwarzania tekstu za pomocą stemingu jest:

```
bieganie -> bieg  
studiować -> studi  
lepsz -> lepsz
```

1.2.3. Lematyzacja

Lematyzacja jest podejściem, które stanowi alternatywę dla stemmingu. Główna idea polega na sprowadzeniu słowa do formy słownikowej – lematu. Przykładem lematyzacji dla języka polskiego jest sprowadzenie rzeczowników do mianownika liczby pojedynczej, przymiotników do mianownika liczby pojedynczej rodzaju męskiego, a czasowników, imiesłów i rzeczowników odczasownikowych do formy bezokolicznika.

Przykładem przetwarzania tekstu za pomocą lematyzacji jest:

biegający -> biegać
studiów -> studium
lepszemu -> dobry

1.3. Wektoryzacja

Większość modeli matematycznych działa w przestrzeniach wektorowych o dużych wymiarach, dlatego konieczne jest przekształcenie tekstu w przestrzeń wektorową. Głównym celem wektoryzacji tekstu jest stworzenie reprezentacji danych tekstowych, która zachowuje informacje semantyczne i syntaktyczne o tekście, a jednocześnie nadaje się do wykorzystania w modelach matematycznych. Pozwala to na efektywną analizę i przetwarzanie danych tekstowych za pomocą algorytmów i metod uczenia maszynowego [4].

1.3.1. Cele wektoryzacji

Główne cele wektoryzacji tekstu obejmują reprezentowanie tekstu do analizy przez algorytmy maszynowe, znajdowanie podobieństw między tekstami lub kategoryzowanie tekstów według tematyki. Wektoryzacja tekstu jest również wykorzystywana do rozwiązywania zadań klasyfikacji, klasteryzacji lub regresji na podstawie danych tekstowych. Dodatkowo, umożliwia tworzenie modeli uczenia maszynowego do automatycznego uogólniania informacji tekstowych.

1.3.2. Metody wektoryzacji

Pierwszym krokiem podczas wektoryzacji tekstu jest określenie przestrzeni cech, w której tekst będzie reprezentowany jako wektory. Każda cecha może reprezentować słowo, frazę, symbol lub inny element tekstu. Istnieje kilka metod wektoryzacji, w tym:

- **Bag of Words (BoW)** – każdy dokument jest reprezentowany jako wektor, gdzie każdy element odpowiada poszczególnemu słowu, a wartościami są częstotliwości występowania słów w dokumencie,
- **TF-IDF (Term Frequency-Inverse Document Frequency)** – metoda ta uwzględnia nie tylko częstotliwość występowania słów w dokumencie (TF), ale także odwrotną częstotliwość występowania słowa we wszystkich dokumentach (IDF), co pozwala wyróżnić słowa kluczowe,
- **Word Embeddings** – są to metody oparte na trenowaniu sieci neuronowych, które przekształcają słowa w wektory w przestrzeni ciągłej, zachowując ich właściwości semantyczne.

1.3.3. Zastosowanie

Wektoryzacja tekstu znajduje zastosowanie w wielu dziedzinach. Jest używana do analizy nastrojów i sentymentów w mediach społecznościowych oraz recenzjach. Ponadto, wykorzystywana jest do klasyfikacji tekstów według tematyki lub kategorii oraz w systemach rekomendacyjnych opartych na opisach tekstowych. Dodatkowo, odgrywa istotną rolę w automatycznym tłumaczeniu tekstu i implementacji chatbotów.

1.4. Redukcja wymiarowości

Redukcja wymiarowości to proces zmniejszania liczby cech (wymiarów) w zbiorze danych, przy jednoczesnym zachowaniu jak największej ilości informacji. Często stosowana jest w uczeniu maszynowym, gdzie głównym celem jest eliminacja nadmiarowych, nieinformatywnych lub mało istotnych cech, które mogą obniżać skuteczność modelu. Po takim przekształceniu model staje się prostszy, co zmniejsza rozmiar zbioru danych w pamięci i przyspiesza działanie algorytmów. Redukcja wymiarowości jest również używana do uproszczenia analizy i wizualizacji danych, na przykład poprzez redukcję do niskich wymiarów takich jak 2D lub 3D [5].

1.4.1. Metody redukcji wymiarowości

Redukcja wymiarowości może być realizowana metodami wyboru cech (ang. feature selection) lub wydobywania cech (ang. feature extraction).

Wybór cech

Metody wyboru cech pozostawiają pewne podzbiory oryginalnego zestawu cech, eliminując cechy nadmiarowe i mało informatywne. Główne zalety tego rodzaju algorytmów to zmniejszenie prawdopodobieństwa przeuczenia, zwiększenie dokładności predykcji modelu, skrócenie czasu uczenia oraz zwiększenie semantycznego zrozumienia modelu.

Wydobywanie cech

Innym sposobem zmniejszenia wymiarowości danych wejściowych jest wydobywanie cech. Te metody tworzą z oryginalnych cech nowe, które nadal w pełni opisują przestrzeń zbioru danych, ale zmniejszają jego wymiarowość, tracąc na reprezentatywności danych, ponieważ staje się niejasne, za co odpowiadają nowe cechy. Metody wydobywania cech można podzielić na liniowe i nieliniowe.

Metody wydobywania cech

Liniowe metody opierają się na założeniu, że zależność między zmiennymi niezależnymi a zmienną zależną jest liniowa. Oznacza to, że zmiana jednej zmiennej prowadzi do proporcjonalnej zmiany innej. Jedną z najbardziej znanych metod liniowego wydobywania cech jest **PCA (Principal Component Analysis)**. Główną ideą tej metody jest znalezienie hiperpłaszczyzny (płaszczyzny o wyższej liczbie wymiarów), na którą przy ortogonalnej projekcji (projekcji pod kątem prostym) wszystkich cech maksymalizowana jest wariancja (czyli rozproszenie danych).

Nieliniowe metody pozwalają modelować bardziej złożone i nieliniowe zależności między zmiennymi. **Algorytm t-SNE (t-distributed Stochastic Neighbor Embedding)** stara się zachować względne odległości między punktami w oryginalnej przestrzeni wysokowymiarowej, a następnie próbuje odtworzyć ten rozkład w przestrzeni niskowymiarowej.

1.4.2. Zastosowanie

Metoda redukcji wymiarowości jest często i skutecznie stosowana w uczeniu maszynowym, przetwarzaniu obrazów i widzeniu komputerowym, bioinformatyce i genomice, analizie finansowej i handlu oraz wielu innych dziedzinach. Zmniejszenie liczby cech pozwala na optymalizację przetwarzania danych.

Rozdział 2

Wykorzystane metody i technologie

W tym rozdziale omówiono wybrane metody i technologie dla projektu. Omówiono etapy wyboru narzędzi i technologii dla aplikacji graficznej, która umożliwia importowanie danych tekstowych z YouTube, wstępne przetwarzanie tekstu, wektoryzację, grupowanie oraz wizualizację tekstu z elastycznymi ustawieniami parametrów i metod.

2.1. Opis implementacji

Ten projekt ma na celu stworzenie narzędzia do analizy danych tekstowych, obejmującego wstępne przetwarzanie tekstu, wektoryzację i klasteryzację. Głównym celem aplikacji jest umożliwienie użytkownikowi elastycznego dostosowania i wykorzystania różnych metod przetwarzania danych do rozwiązywania konkretnych zadań.

2.1.1. Zastosowane technologie

Do realizacji projektu wybrano język programowania Python (w omawianym projekcie w wersji 3.10.2). Posiada on szereg zalet, takich jak bogaty zestaw bibliotek do implementacji różnych metod, w tym wstępnego przetwarzania tekstu, wektoryzacji i klasteryzacji. Python cechuje się wysoką efektywnością w analizie danych dzięki bibliotekom takim jak NumPy (w omawianym projekcie w wersji 1.26.4) oraz scikit-learn (w omawianym projekcie w wersji 1.4.2). Dodatkowo, społeczność i dokumentacja Pythona ułatwiają rozwój i utrzymanie projektów.

2.1.2. Technologie tworzenia interfejsów graficznych

Główną ideą aplikacji jest stworzenie wygodnego narzędzia do analizy danych tekstowych, w którym użytkownik może samodzielnie wybierać i dostosowywać metody przetwarzania do konkretnych zadań za pomocą interfejsu graficznego. Do opracowania interfejsu graficznego wykorzystano biblioteki PyQt i Matplotlib [17, 18]:

- **PyQt 5.15.10** umożliwia tworzenie intuicyjnych interfejsów użytkownika,
- **Matplotlib 3.9.0** stosuje się wykorzystywany do wizualizacji danych.

Interfejs graficzny umożliwia użytkownikowi ładowanie danych, konfigurowanie parametrów przetwarzania tekstu, wektoryzacji i klasteryzacji oraz wizualizację wyników.

2.1.3. Struktura aplikacji

Głównym komponentem niniejszej aplikacji jest klasa **Workspace**, która stanowi kontener przechowujący parametry dla trzech głównych procesów.

- **Przetwarzanie tekstu** obejmuje metody wstępnego przetwarzania, takie jak tokenizacja, usuwanie stop-words i lematyzacja,
- **Wektoryzacja** przekształca dane tekstowe w wektory liczbowe przy użyciu metod takich jak TF-IDF lub Word2Vec,
- **Klasteryzacja** implementuje algorytmy, takie jak K-means lub DBSCAN do grupowania danych tekstowych.

Klasa **Workspace** zapewnia strukturalne przechowywanie i zarządzanie parametrami każdego etapu, co upraszcza proces konfiguracji i wykonania analizy.

2.2. Dane tekstowe

Aby realizować ten projekt, potrzebne są dane tekstowe, które w tym przypadku są komentarzami z YouTube. Do pozyskiwania informacji o filmach i powiązanych komentarzach używane jest YouTube Data API v3. API to jest darmowe i łatwe w użyciu. Aby rozpocząć pracę należy zarejestrować się i utworzyć projekt w Google Cloud, a następnie aktywować YouTube Data API. Po tych krokach wygenerowany zostanie klucz API, który jest potrzebny do uwierzytelniania.

2.2.1. Zapytania API

W projekcie wykorzystane są dwa zapytania HTTP:

GET <https://www.googleapis.com/youtube/v3/commentThreads>

GET <https://www.googleapis.com/youtube/v3/videos>

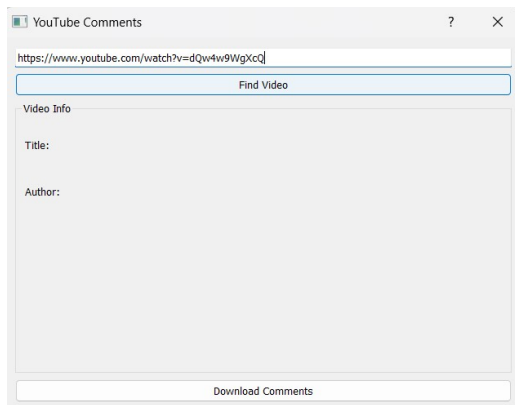
Pierwsze zapytanie służy do pobierania komentarzy do konkretnego filmu, a drugie do uzyskiwania informacji o filmie. Szczegółowe informacje można znaleźć w dokumentacji [8]. Krokami pobierania komentarzy są:

1. Użytkownik wprowadza w interfejsie graficznym (GUI) link do interesującego go filmu. W tym momencie interfejs graficzny wygląda jak na grafiku 2.1
2. Program wyodrębnia **video_id** z podanego linku.
3. Za pomocą **video_id** wykonywane jest zapytanie **GET <https://www.googleapis.com/youtube/v3/videos>**, aby uzyskać podstawowe informacje o filmie. To pozwala użytkownikowi upewnić się, że wybrany film odpowiada jego oczekiwaniom. W tym momencie interfejs graficzny prezentuje się jak na grafiku 2.2
4. Po potwierdzeniu przez użytkownika, że film jest prawidłowy, wykonywane jest zapytanie **GET <https://www.googleapis.com/youtube/v3/commentThreads>**, aby pobrać komentarze do tego filmu.
5. Uzyskane komentarze są zapisywane w pliku tekstowym, gdzie każdy komentarz jest zapisany w oddzielnej linii. Nazwa pliku odpowiada unikalnemu identyfikatorowi filmu (**video_id**).

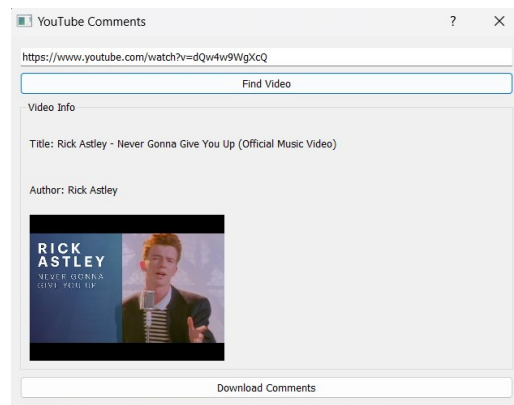
2.2.2. Przetwarzanie uzyskanych danych

Wyniki naszych zapytań są dostarczane w formacie JSON. Szczegółowe informacje o strukturze odpowiedzi JSON można znaleźć w dokumentacji [8]. Do uzyskiwania informacji o filmie używane są następujące pola:

- **title**: tytuł filmu,
- **channelTitle**: nazwa autora kanału,
- **thumbnails**: URL miniaturki.



Rysunek 2.1: Interfejs graficzny po wprowadzeniu linku do filmu (krok 1)



Rysunek 2.2: Interfejs graficzny po uzyskaniu informacji o filmie (krok 3)

Przy pobieraniu komentarzy potrzebne są następujące pola:

- **textDisplay**: treść komentarza,
- **replies**: kontener z odpowiedziami na komentarze.

Pole **textDisplay** zawiera tekst komentarza, a pole **replies** jest kontenerem na odpowiedzi na komentarze. Każda odpowiedź w **replies.comments** również ma pole **textDisplay**, które możemy osobno przetwarzać. W końcowym wyniku otrzymamy listę wszystkich komentarzy, gdzie każdy element reprezentuje oddzielny komentarz.

2.2.3. Przechowywanie

Po uzyskaniu odpowiedzi wszystkie komentarze kierują do osobnego katalogu **comments**. Katalog zostanie utworzony, jeśli nie istnieje. Każdy plik z komentarzami ma nazwę **video_id.txt** dla łatwej identyfikacji.

Dla uproszczenia procesu przechowywania i dostępu do danych oraz zapewnienia ich kompatybilności z różnymi narzędziami analitycznymi, zdecydowano się na użycie zwykłego pliku tekstowego (.txt) do przechowywania komentarzy. Każdy komentarz jest zapisany w oddzielnej linii pliku.

Taki sposób przechowywania danych został wybrany z kilku powodów. Pierwszym z nich jest prostota implementacji, gdyż pliki tekstowe łatwo tworzyć, czytać i edytować za pomocą wielu narzędzi programistycznych i języków programowania. Ponadto kompatybilność jest istotna, ponieważ praktycznie wszystkie narzędzia analityczne i biblioteki mogą pracować z plikami tekstowymi, co ułatwia integrację danych w różnych etapach analizy. Wreszcie, mały wpływ na analizę wynika z tego, że format przechowywania nie ma znaczącego wpływu na sam proces analizy danych; główną wagę ma zawartość komentarzy, a nie sposób ich przechowywania.

2.3. Przetwarzania tekstu

Następnym procesem jest przetwarzanie tekstu. Ważne jest, aby zrozumieć, jak wyglądają komentarze po ich pobraniu i zapisaniu z YouTube. Implementacja przetwarzania zostanie wykonana za pomocą biblioteki NLTK (Natural Language Toolkit) w wersji 3.8.1 oraz wbudowanych funkcji Pythona. Szczegóły biblioteki NLTK można znaleźć w dokumentacji [9]. Przetwarzanie tekstu jest podzielone na kilka etapów:

- przetwarzanie obowiązkowe, które jest wykonywane niezależnie od wybranych parametrów przez użytkownika,

- przetwarzanie prymitywne, czyli usuwanie symboli i cyfr,
- usuwanie stop-words,
- lematyzacja,
- analiza.

Wszystkie metody, oprócz przetwarzania obowiązkowego i analizy, są opcjonalne, co oznacza, że wykonanie zacznie się tylko wtedy, gdy użytkownik wskaże, że chce używać tych metod.

2.3.1. Przetwarzanie obowiązkowe

Obowiązkowe etapy przetwarzania tekstu to zmiana wszystkich wielkich liter na małe we wszystkich słowach oraz proces tokenizacji, który eliminuje potrzebę usuwania spacji. Za pomocą funkcji `nltk.word.tokenize` każde słowo zostanie podzielone na tokeny. Te etapy są obowiązkowe, ponieważ upraszczają dalszą pracę z tekstem i gwarantują poprawne działanie wielu późniejszych funkcji, które mogą być czułe na wielkość liter.

2.3.2. Przetwarzanie prymitywne

Przetwarzanie prymitywne polega na usuwaniu niealfabetycznych symboli lub słów. Ten proces nie jest obowiązkowy, jednak jego zastosowanie pomaga pozbyć się różnych języków znaczników, które YouTube używa do wizualizacji emotikon lub narzędzi do zmiany tekstu.

Przykładem takich komentarzy są:

```
Great video! :) :(
Check this out: <a href='https://example.com'>link</a>
```

W tym projekcie jest to realizowane za pomocą wbudowanej funkcji Pythona `isalpha()`, która sprawdza, czy symbole lub słowa zawierają niealfabetyczne znaki, jak widać na Listingu 2.1.

```
1 def remove_non_alpha(tokens):
2     return [word for word in tokens if word.isalpha()]
```

Listing 2.1: Funkcja usuwająca niealfabetyczne symbole

2.3.3. Usuwanie stop-words

Etap usuwania stop-words polega na usuwaniu słów, które nie niosą znaczącej informacji. Typowymi przykładami takich słów są:

`the, is, in, and`

Usuwanie stop-words zmniejsza rozmiar tekstu i sprawia, że znaczące słowa są bardziej widoczne. Do tego celu używa się słownika stop-words z biblioteki NLTK, który w razie potrzeby można uzupełnić własnymi słowami. Przykład implementacji znajduje się na Listingu 2.2.

```
1 from nltk.corpus import stopwords
2
3 def remove_stop_words(tokens):
4     stop_words = set(stopwords.words('english'))
5     return [word for word in tokens if word not in stop_words]
```

Listing 2.2: Funkcja usuwająca stop-words

2.3.4. Lematyzacja

Ostatnim opcjonalnym procesem jest lematyzacja, która sprowadza słowa do ich podstawowej formy. Jest to przydatne w sytuacjach, gdy trzeba zgrupować różne formy jednego słowa, na przykład:

running, ran, runs do run

Przykład implementacji znajduje się na Listingu 2.3.

```
1 from nltk.stem import WordNetLemmatizer
2
3 def lemmatize_tokens(tokens):
4     lemmatizer = WordNetLemmatizer()
5     return [lemmatizer.lemmatize(word) for word in tokens]
```

Listing 2.3: Funkcja lematyzująca tokeny

2.3.5. Analiza

Na końcu, niezależnie od wybranych przez użytkownika etapów, wszystkie tokeny są sprawdzane pod kątem niepustych wartości, aby uniknąć całkowicie pustych słów i wierszy. Następnie wszystkie tokeny są łączone w jeden ciąg znaków, oddzielony spacjami. Na tym etapie wynik przetwarzania jest zwracany i analizowany za pomocą funkcji analizy częstotliwości (**freq_analysis**) i analizy długości (**length_analysis**). Te funkcje są używane do tworzenia wykresów, które pomogą użytkownikowi wizualnie ocenić częstotliwość występowania słów i długość komentarzy w celu podejmowania dalszych decyzji.

Analiza częstotliwości

Analiza częstotliwości (frequency analysis) używa przetworzonych ciągów znaków, tworząc pojedynczą tablicę ze wszystkimi słowami ze wszystkich komentarzy i stosuje klasę **collections.Counter** z języka Python. **Counter** jest używany do zliczania hashowalnych obiektów, takich jak ciągi znaków lub liczby i dostarcza wygodne metody do pracy z danymi częstotliwości. W ten sposób funkcja generuje dwie tablice: tablicę słów i odpowiadającą jej tablicę liczby wystąpień każdego słowa w tekście. Te tablice są używane do tworzenia wykresu słupkowego, pokazującego częstotliwość występowania słów.

Przykład implementacji algorytmu w projekcie znajduje się na Listingu 2.4.

```
1 from collections import Counter
2
3 def freq_analysis(tokens):
4     data = ' '.join(self.nlp.data)
5     words = nltk.word_tokenize(data)
6     word_freq = Counter(words)
7     words_counts_sorted = word_freq.most_common()
8     self.words_sorted = [word_count[0] for word_count in words_counts_sorted]
9     self.counts_sorted = [word_count[1] for word_count in words_counts_sorted]
```

Listing 2.4: Funkcja przeprowadzająca analizę częstotliwości tokenów

Analiza długości

Analiza długości (length analysis) również używa przetworzonych ciągów znaków komentarzy, określając liczbę słów w każdym komentarzu. Ostatecznie generowana jest tablica z liczbą słów, która jest używana do tworzenia histogramu, pokazującego rozkład długości komentarzy.

Przykład implementacji algorytmu w projekcie znajduje się na Listingu 2.5.

```
1
2 def length_analysis(tokens):
3     word_counts = [len(nltk.word_tokenize(el)) for el in self.nlp.data]
4     self.max_size = max(word_counts) + 1
5     self.lengths = [0] * (self.max_size + 1)
6     for count in word_counts:
7         self.lengths[count] += 1
8     endTime = time.time()
```

Listing 2.5: Funkcja przeprowadzająca analizę długości tokenów

2.4. Wektoryzacja

Na tym etapie tekst jest gotowy do dalszego przetwarzania. Aby przeprowadzić analizę, konieczne jest przekształcenie komentarzy w formę wektorową (wektoryzacja). Zostaną użyte dwie metody: TF-IDF i Word2Vec. Te metody umożliwiają przedstawienie danych tekstowych w postaci wektorów liczbowych. Wektoryzacja jest realizowana przy użyciu bibliotek **scikit-learn** [10] dla metody TF-IDF oraz **gensim** [11] dla metody Word2Vec. Metody zostały wybrane ze względu na swoją popularność, a wybór dwóch metod wynika z potrzeby przeprowadzenia porównań.

2.4.1. TF-IDF

TF-IDF (Term Frequency-Inverse Document Frequency) to miara statystyczna używana do oceny ważności terminu w dokumencie w odniesieniu do kolekcji dokumentów (korpusu). Jest szeroko stosowana w zadaniach przetwarzania języka naturalnego i wyszukiwania informacji. W bibliotece **scikit-learn** istnieje wygodne narzędzie do obliczania TF-IDF: klasa **TfidfVectorizer** [12].

Parametry

Do konfiguracji algorytmu TF-IDF używane są następujące parametry:

- **max_df (liczba lub udział procentowy)**: Terminy, które występują w większej liczbie dokumentów niż podano są ignorowane,
- **min_df (liczba lub udział procentowy)**: Terminy, które występują w mniejszej liczbie dokumentów niż podano, są ignorowane,
- **ngram_range (min_n, max_n)**. Określa zakres wyodrębniania n-gramów. N-gramy to ciągi N kolejnych elementów (zwykle słów lub znaków) z danego tekstu,
- **stop_words**: Lista słów lub ciąg znaków określający słowa, które należy ignorować.

Algorytm TF-IDF

W tej sekcji omówione są kluczowe miary analizy tekstu: Term Frequency (TF), Inverse Document Frequency (IDF) oraz ich kombinacja: TF-IDF.

- **Term Frequency (TF)** – miara częstości występowania danego słowa w dokumencie. TF jest obliczane jako liczba wystąpień danego słowa (termu) w dokumencie podzielona przez całkowitą liczbę słów w dokumencie. To pozwala określić, jak ważne jest słowo w danym dokumencie.
- **Inverse Document Frequency (IDF)** – miara, która ocenia, jak rzadko lub często występuje dane słowo we wszystkich dokumentach w korpusie. IDF jest obliczane jako logarytm dziesiętny z odwrotnego stosunku liczby dokumentów zawierających term do całkowitej liczby dokumentów w korpusie. IDF pomaga wyróżnić słowa, które są istotne dla jednego dokumentu w kontekście wszystkich dokumentów.
- **TF-IDF** – kombinacja TF i IDF, używana do oceny ważności danego słowa w kontekście jednego dokumentu w całym korpusie. TF-IDF dla danego termu w dokumencie można obliczyć jako iloczyn TF i IDF. TF-IDF umożliwia określenie, jak ważne jest słowo w danym dokumencie w porównaniu do innych dokumentów w korpusie. Słowa o wysokim TF-IDF są często unikalne dla danego dokumentu i mogą służyć jako kluczowe pojęcia lub cechy w analizie tekstu.

Przykład użycia

Przykład użycia metody TF-IDF znajduje się na Listingu 2.6.

```
1 from sklearn.feature_extraction.text import TfidfVectorizer
2
3
4 def vectorize_tfidf(corpus):
5     vectorizer = TfidfVectorizer()
6     tfidf_matrix = vectorizer.fit_transform(corpus)
7     return tfidf_matrix
```

Listing 2.6: Funkcja wektoryzująca korpus za pomocą TF-IDF

Przykładem przekształcenia danych za pomocą TF-IDF jest:

Przed: ["This is a great video", "I love this content"]
Po: [[0.5, 0.5, 0, 0.5, 0, 0.5], [0, 0, 0.7, 0, 0.7, 0]]

Aby korzystać z TF-IDF potrzebny jest korpus tekstów, który można przedstawić jako listę ciągów znaków, gdzie każdy ciąg znaków to osobny dokument.

2.4.2. Word2Vec

Word2Vec to algorytm opracowany do nauki wektorowych reprezentacji słów (embeddingów), które oddają semantyczne relacje między słowami. W bibliotece **gensim** dostępny jest wygodny interfejs do pracy z Word2Vec [13].

Parametry

Do konfiguracji algorytmu Word2Vec używane są następujące parametry:

- **sentences**: korpus tekstów, przedstawiony jako lista list słów,
- **vector_size**: wymiarowość wektorowej reprezentacji słów,
- **window**: maksymalna odległość między bieżącym a przewidywanym słowem w zdaniu,
- **min_count**: minimalna częstotliwość słowa w korpusie, aby je uwzględnić,
- **workers**: liczba wątków do równoległego uczenia.

Dodatkowe ustawienia:

- **SG (Skip-Gram) i CBOW (Continuous Bag of Words)**: W Word2Vec są dwa główne podejścia architektoniczne – Skip-Gram i CBOW. Parametr **sg** w **gensim** pozwala wybrać pomiędzy nimi:
 - **sg=0** używa CBOW (domyślnie),
 - **sg=1** używa Skip-Gram.
- **negative** – określa liczbę *negatywnych* przykładów dla każdego pozytywnego przykładu,
- **alpha** – określa początkową szybkość uczenia, a **min_alpha** — minimalną szybkość uczenia, która zmniejsza się do tej wartości w trakcie uczenia.

Algorytm Word2vec

W tej sekcji omówiono algorytm Word2Vec oraz jego dwie główne warianty: Skip-Gram i CBOW.

1. Inicjalizacja wag: Wagi sieci neuronowej są inicjalizowane losowymi wartościami. Są to wagi między warstwą wejściową a ukrytą oraz między warstwą ukrytą a wyjściową.

2. Przejście do przodu (Forward Pass): Dla każdego słowa w zdaniu tworzony jest wektor wejściowy. W przypadku Skip-Gram ten wektor wejściowy jest używany do przewidywania słów kontekstowych. W przypadku CBOW słowa kontekstowe są używane do przewidywania bieżącego słowa.
 - Skip-Gram:
 - słowo wejściowe jest kodowane jako jednowymiarowy wektor,
 - jednowymiarowy wektor jest mnożony przez wagi między warstwą wejściową a ukrytą, aby uzyskać ukrytą reprezentację,
 - ukryta reprezentacja jest mnożona przez wagi między warstwą ukrytą a wyjściową, aby przewidzieć słowa kontekstowe.
 - CBOW:
 - słowa kontekstowe są kodowane jako jednowymiarowe wektory,
 - jednowymiarowe wektory są sumowane i mnożone przez wagi między warstwą wejściową a ukrytą, aby uzyskać ukrytą reprezentację,
 - ukryta reprezentacja jest mnożona przez wagi między warstwą ukrytą a wyjściową, aby przewidzieć bieżące słowo.
3. Propagacja wsteczna (Backpropagation): Błąd przewidywania jest obliczany przez porównanie przewidywanego słowa z faktycznym. Następnie ten błąd jest propagowany wstecz przez sieć, aby zaktualizować wagi. Używa się metody stochastycznego spadku gradientu (SGD) i, w niektórych przypadkach, negatywnego próbkowania, aby przyspieszyć uczenie.
 - Aktualizacja wag: Wagi są aktualizowane w kierunku przeciwnym do gradientu błędu. To pomaga zminimalizować błąd przewidywania.
4. Powtarzanie procesu: Proces ten powtarza się dla wszystkich słów w zdaniu i dla wszystkich zdań w korpusie. Model przechodzi przez korpus kilka razy, zgodnie z wartością parametru **epochs**.

Przykład użycia

Przykład użycia metody Word2Vec znajduje się na Listingu 2.7.

```

1 from gensim.models import Word2Vec
2
3 def vectorize_word2vec(tokens):
4     model = Word2Vec(sentences=tokens, vector_size=100, window=5, min_count=1, workers=4)
5     word_vectors = model.wv
6     return word_vectors

```

Listing 2.7: Funkcja wektoryzująca za pomocą Word2Vec

Przykładem przekształcenia danych za pomocą Word2vec jest:

```

Przed: ["This is a great video", "I love this content"]
Po: Wektory słów: { "this": [...], "is": [...], "a": [...], "great": [...],
"video": [...], "I": [...], "love": [...], "content": [...] }

```

2.5. Klasteryzacja

Kolejnym etapem przetwarzania danych jest klasteryzacja, która wykorzystuje zwektoryzowane komentarze i dzieli je na klastry. Zostanie to realizowane za pomocą dwóch metod: K-means i DBSCAN, z wykorzystaniem biblioteki **sklearn** [10, 15].

Dla możliwości porównywania wybrano dwie z najpopularniejszych metod.

2.5.1. K-means

Algorytm K-means dzieli zbiór danych na nieprzekrywające się klastry, z których każdy opisany jest średnią wartością (centroidem) próbek w klastrze. K-means dąży do wyboru centroidów, minimalizując bezwładność, czyli kryterium sumy kwadratów wewnątrz klastrów [14].

Parametry

Do konfiguracji algorytmu K-means używane są następujące parametry:

- **n_clusters** – liczba klastrów (K), na które należy podzielić dane,
- **init** – metoda inicjalizacji centroidów,
- **max_iter** – maksymalna liczba iteracji dla jednego uruchomienia K-means,
- **tol** – próg dla zatrzymania algorytmu. Jeśli różnica między starymi a nowymi centroidami jest mniejsza od tej wartości, algorytm zatrzymuje się,
- **n_init** – liczba uruchomień algorytmu z różnymi inicjalizacjami. Wynik z najmniejszą bezwładnością zostanie wybrany jako ostateczny,
- **random_state** – wartość początkowa generatora liczb losowych dla powtarzalności wyników,
- **algorithm** – algorytm do obliczania K-means. Możliwe wartości to **auto**, **full** i **elkan**.

Algorytm K-means

Algorytm K-means można zrozumieć przez trzy główne kroki:

1. **Wybór początkowych centroidów** – algorytm rozpoczyna od wyboru początkowych centroidów. Zazwyczaj stosuje się prostą metodę losowego wyboru próbek z zestawu danych.
2. **Przypisanie do klastrów** – każda próbka danych jest przypisywana do najbliższego centroidu na podstawie odległości euklidesowej.
3. **Aktualizacja centroidów** – nowe położenie centroidów jest obliczane na podstawie średniej wartości wszystkich próbek przypisanych do każdego z klastrów.

Proces ten powtarza się, dopóki różnica między starymi a nowymi centroidami nie będzie mniejsza od określonego progu. Algorytm K-means jest podatny na problem lokalnych minimów, co zależy od początkowej inicjalizacji centroidów. Zazwyczaj uruchamia się algorytm kilkakrotnie z różnymi początkowymi wartościami centroidów, aby poprawić jego skuteczność.

Pseudokod algorytmu

Pseudokod implementujący algorytm K-means znajduje się na Listingu 2.8.

```
1 1. Wybierz losowo k punktów jako początkowe centroidy
2 2. Dla każdej próbki w danych:
3   a. Przypisz próbkę do najbliższego centroidu
4 3. Powtarzaj aż do spełnienia warunku stopu:
5   a. Dla każdego centroidu:
6     i. Oblicz nową średnią wartość wszystkich próbek przypisanych do tego centroidu
7     b. Oblicz różnicę między starymi a nowymi centroidami
8     c. Jeśli różnica jest mniejsza niż określony próg, zakończ iterację
9     d. Przypisz nowe wartości centroidów do starych centroidów
10  e. Wyczyść przypisanie próbek do centroidów
11 4. Zwróć przypisanie próbek do centroidów oraz finalne wartości centroidów
```

Listing 2.8: Pseudokod algorytmu K-means

Przykład użycia

Przykład użycia metody K-means znajduje się na Listingu 2.9.

```
1 kmeans = KMeans(n_clusters=2, init='k-means++', max_iter=300, n_init=10, random_state=0)
2 kmeans.fit(X)
3 return kmeans.labels_
```

Listing 2.9: Przykład użycia algorytmu K-Means

2.5.2. DBSCAN

Algorytm DBSCAN traktuje klastry jako obszary o wysokiej gęstości, oddzielone od obszarów o niskiej gęstości. Dzięki temu DBSCAN może wykrywać klastry o dowolnym kształcie, w przeciwieństwie do K-means, który zakłada, że klastry mają kształt wypukły. Głównym elementem DBSCAN jest pojęcie punktów rdzeniowych, które reprezentują próbki w obszarach o wysokiej gęstości [15].

Parametry

Do konfiguracji algorytmu DBSCAN używane są następujące parametry:

- **eps** – maksymalna odległość między dwiema próbkami, przy której jedna jest uważana za sąsiada drugiej. Jest to parametr promienia do określania gęstości obszaru.
- **min_samples** – minimalna liczba próbek (w tym sama próbka) potrzebna do uznania obszaru za gęsty, tj. do uznania próbki za rdzeniową.
- **metric** – metryka do obliczania odległości między punktami (domyślnie **euclidean**). Może to być dowolna metryka obsługiwana przez funkcję **scipy.spatial.distance.pdist**.
- **algorithm** – algorytm używany do obliczania najbliższych sąsiadów (**auto**, **ball_tree**, **kd_tree**, **brute**).
- **leaf_size** – rozmiar liścia przekazywany algorytmom **ball_tree** i **kd_tree**. Może to wpłynąć na szybkość budowy i zapytania najbliższych sąsiadów.
- **p** – wykładnik odległości Minkowskiego, gdy używana jest metryka **minkowski** (domyślnie **p=2**, co odpowiada odległości euklidesowej).

Algorytm DBSCAN

Algorytm DBSCAN ma następujące kroki:

1. **Określenie punktów rdzeniowych** – próbki, dla których liczba sąsiadów w obrębie **eps** nie jest mniejsza niż **min_samples**.
2. **Tworzenie klastrów** – dla każdej próbki rdzeniowej tworzony jest klaster poprzez znalezienie wszystkich jej sąsiadów, którzy również są próbkami rdzeniowymi. Następnie dla tych nowo znalezionych próbek rdzeniowych również poszukuje się ich sąsiadów, będących próbkami rdzeniowymi, kontynuując ten proces iteracyjnie. W ten sposób klaster rośnie, obejmując coraz więcej próbek rdzeniowych. Dodatkowo każdy klaster obejmuje także próbki niebędące rdzeniowymi, które są sąsiadami próbek rdzeniowych w klastrze.
3. **Wykrywanie szumów** – wszystkie próbki, które nie są rdzeniowe i nie znajdują się w wystarczającej odległości od próbek rdzeniowych, są uznawane za szumy.

Algorytm DBSCAN jest deterministyczny i zawsze generuje takie same klastry dla tych samych danych wejściowych w tej samej kolejności. Jednak wyniki mogą się różnić przy podaniu danych w innej kolejności. Wynika to z faktu, że kolejność przetwarzania danych wpływa na sposób, w jaki próbki są przypisywane do klastrów.

Pseudokod algorytmu

Pseudokod implementujący algorytmu DBSCAN znajduje się na Listingu 2.10.

```
1 1. Utwórz pusty zbiór klastrów
2 2. Dla każdej próbki $P$ w danych:
3   a. Oblicz odległość od wszystkich innych próbek
4   b. Znajdź wszystkie sąsiednie próbki w obrębie epsilon (promień sąsiedztwa)
5   c. Jeśli liczba sąsiadów jest większa lub równa min_samples, dodaj próbkę P do zbioru punktów
6     rdzeniowych
7 3. Dla każdej próbki rdzeniowej:
8   a. Utwórz nowy klastę i dodaj do niego próbkę rdzeniową
9   b. Dla każdej sąsiedniej próbki rdzeniowej:
10    i. Jeśli nie została jeszcze przypisana do żadnego klastra, dodaj ją do bieżącego klastra
11    j. Jeśli jest próbką rdzeniową, dodaj jej sąsiadów do bieżącego klastra
4. Zwróć zbiór klastrów i punkty uznane za szum
```

Listing 2.10: Pseudokod algorytmu DBSCAN

Przykład użycia

Przykład użycia metody DBSCAN znajduje się na Listingu 2.11.

```
1 dbscan = DBSCAN(eps=0.1, min_samples=4)
2
3 dbscan.fit(X)
4 return dbscan.labels_
```

Listing 2.11: Przykład użycia algorytmu DBSCAN

2.5.3. Dobór parametrów

Dobór odpowiednich parametrów dla algorytmów klasteryzacji, takich jak K-means i DBSCAN jest kluczowy dla uzyskania satysfakcjonujących wyników. Poniżej przedstawione są niektóre techniki i wskazówki dotyczące doboru parametrów:

- **K-means:** Ważnym parametrem jest liczba klastrów (**n_clusters**). Można użyć metody *łokcia* (elbow method) lub średniego współczynnika sylwetki (silhouette score) do określenia optymalnej liczby klastrów. Parametry takie jak **max_iter**, **tol** oraz **n_init** mogą być dostosowane w celu poprawy zbieżności i stabilności algorytmu.
- **DBSCAN:** Kluczowymi parametrami są **eps** i **min_samples**. Wybór **eps** zależy od gęstości danych i można go oszacować na podstawie wykresu odległości K-najbliższych sąsiadów (K-distance plot). Parametr **min_samples** można ustawić w zależności od oczekiwanej minimalnej liczby punktów w klastrze.

2.6. Wizualizacja

Ostatnim etapem analizy jest wizualizacja danych. Końcowy wynik to wykres 2D, gdzie każdy punkt będzie reprezentował pewne dane tekstowe (komentarze) z określonym kolorem odpowiadającym klastrowi, do którego należy lub kolorem braku klastra, jak w przypadku DBSCAN. Na wykresie jest również wizualizowane centroidy każdego klastra w postaci czerwonego krzyżyka.

2.6.1. Redukcja wymiarowości

Aby możliwa była wizualizacja danych jako punktów 2D, konieczne jest skorzystanie z metody redukcji wymiarowości (t-SNE). Ta metoda jest nieliniową metodą ekstrakcji cech. Na początku trudno jest określić, za co odpowiadają cechy, więc wybór odpowiednich cech jest niemożliwy. Metoda nieliniowa jest pomocna w zachowaniu nieliniowej zależności [16].

2.6.2. t-SNE

Parametry

Do konfiguracji algorytmu t-SNE używane są następujące parametry:

- **n_components** – liczba wymiarów, na które należy rzutować dane (zwykle 2 lub 3 dla wizualizacji),
- **perplexity** – parametr związany z liczbą najbliższych sąsiadów. Wpływa na równowagę między lokalną a globalną strukturą danych,
- **early_exaggeration** – parametr kontrolujący odległości między punktami na początkowym etapie optymalizacji,
- **learning_rate** – szybkość uczenia się dla spadku gradientowego,
- **n_iter** – liczba iteracji do optymalizacji,
- **metric** – metryka do obliczania odległości między punktami (domyślnie **euclidean**),
- **init** – metoda inicjalizacji (**random** lub **pca**),
- **random_state** – początkowa wartość generatora liczb losowych dla reprodukowalności wyników.

Opis algorytmu

Algorytm t-SNE ma następujące kroki::

1. Przekształcenie odległości na prawdopodobieństwa: Dla każdego punktu obliczane są prawdopodobieństwa Pearsona, które określają prawdopodobieństwo wybrania sąsiedniego punktu jako sąsiada.
2. Określenie docelowych prawdopodobieństw: Na niskowymiarowej projekcji obliczane są prawdopodobieństwa Q, podobne do prawdopodobieństw Pearsona, ale w innym wymiarze.
3. Minimalizacja dywergencji (Kullback-Leibler divergence): Używany jest spadek gradientowy do minimalizacji dywergencji między rozkładami P i Q, co prowadzi do zachowania bliskości punktów.

Przykład użycia

Przykład użycia metody T-SNE znajduje się na Listingu 2.12.

```
1 X = np.array([[0, 1], [1, 1], [2, 1], [3, 1], [4, 1],  
2 [0, 0], [1, 0], [2, 0], [3, 0], [4, 0]])  
3  
4 tsne = TSNE(n_components=2, perplexity=5, n_iter=300, random_state=0)  
5  
6 X_embedded = tsne.fit_transform(X)  
7
```

Listing 2.12: Przykład użycia t-SNE do osadzania danych

Rozdział 3

Funkcje aplikacji

Ten rozdział opisuje interfejs użytkownika, dostępne funkcje oraz sposób interakcji z nimi. Omówiono strukturę okna głównego, poruszanie się po interfejsie w celu wykorzystania potrzebnych funkcji, importowanie danych, wybór parametrów oraz przedstawiono charakterystykę obszaru roboczego aplikacji.

3.1. Opis aplikacji

Graficzna część aplikacji składa się z okna roboczego i przestrzeni roboczej, w której użytkownik ustawia parametry generowania wyników.

3.1.1. Okno pracy

Okno pracy składa się z menu zawierającym dwa pola **File** i **Comments** oraz główną strefą, w której znajduje się menu zakładek. Każda zakładka odpowiada osobnej przestrzeni roboczej **workspace**.

3.1.2. Menu-bar

Pasek menu umożliwia szybką nawigację po oknie pracy w dowolnym momencie. W menu zaimplementowane są następujące funkcje:

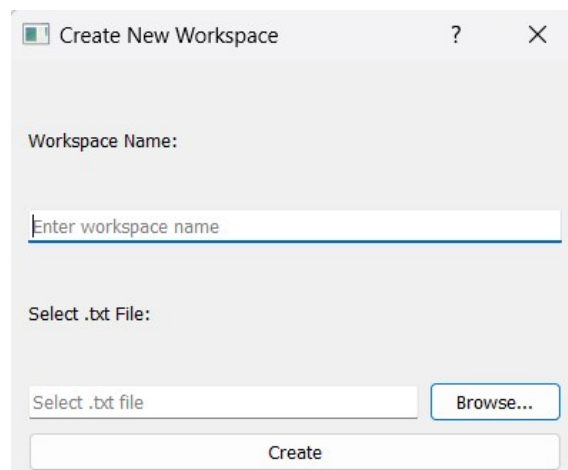
- otwieranie okna dialogowego w celu utworzenia nowej przestrzeni roboczej **File – New Project**,
- otwieranie okna dialogowego w celu pobrania komentarzy z YouTube **Comments – YouTube**.

Tworzenie przestrzeni roboczej

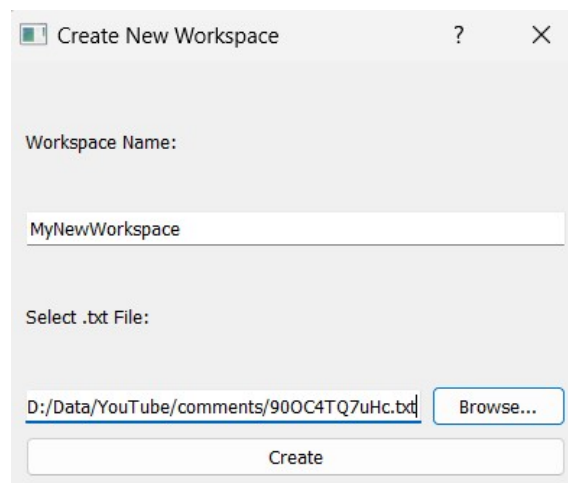
Aby utworzyć nową przestrzeń roboczą, należy podać nazwę przestrzeni roboczej (Workspace name) i plik źródłowy (Source file) zawierający dane tekstowe. Początkowe okno przedstawiono na Rysunku 3.1. Użytkownik musi podać dowolną nazwę dla przestrzeni roboczej i wybrać ścieżkę do pliku tekstowego z danymi. Można to zrobić ręcznie lub wybrać plik za pomocą okna dialogowego `FileDialog` z biblioteki `PyQt`, co pozwala odnaleźć potrzebny plik na komputerze. Przykład wypełnionego poprawnie okna znajduje się na Rysunku 3.2.

Pobieranie komentarzy

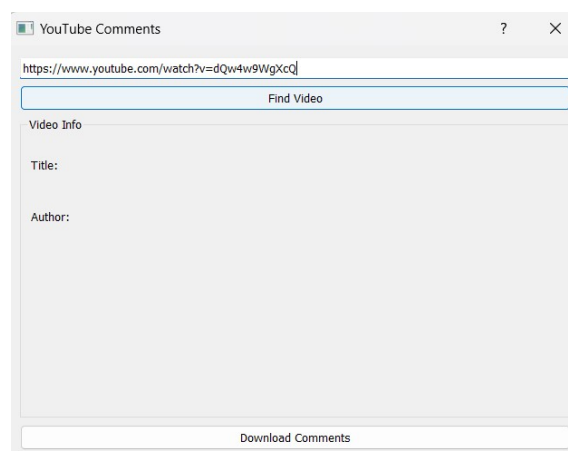
Do pobrania komentarzy służy okno dialogowe, w którym należy podać link do interesującego nas filmu, jak widać na Rysunku 3.3. Jeśli film zostanie znaleziony, w polu Video Info pojawiają się informacje o nim, pozwalając użytkownikowi upewnić się, że to właśnie ten film, komentarze z którego chcemy pobrać. Następnie użytkownik musi kliknąć przycisk `Download comments`, który zapisze wszystkie komentarze.



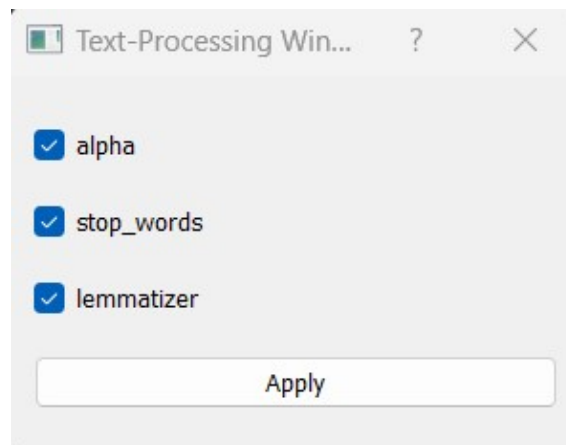
Rysunek 3.1: Przykład początkowego okna



Rysunek 3.2: Przykład wypełnionego okna



Rysunek 3.3: Przykład początkowego okna do pobierania komentarzy



Rysunek 3.4: Przykład okna do konfiguracji przetwarzania tekstu

3.1.3. Workspace

Po utworzeniu przynajmniej jednej przestrzeni roboczej pojawia się menu nawigacyjne zakładek, gdzie każda zakładka odpowiada osobnej przestrzeni roboczej. Można swobodnie przemieszczać się między tymi zakładkami i zamykać je. Zawartość każdej zakładki obejmuje:

- pole na wykresy,
- przyciski **Previous** i **Next** do przemieszczenia wykresów w polu na wykresy,
- przycisk **Text** do otwierania okna dialogowego zarządzania przetwarzaniem tekstu,
- przycisk **Vectorization** do otwierania okna dialogowego zarządzania wektoryzacją,
- przycisk **Cluster** do otwierania okna dialogowego zarządzania klasteryzacją,
- przycisk **Apply** do zastosowania wszystkich wybranych parametrów i otwarcia okna z wynikami klasteryzacji.

Pole na wykresy

Pole na wykresy w każdej chwili może wyświetlać jeden z trzech proponowanych wykresów:

- Wykres częstości słów (**freq_analysis**).
- Wykres długości tekstu (**length_analysis**).
- Wykres najbliższych sąsiadów (**k-nearest**).

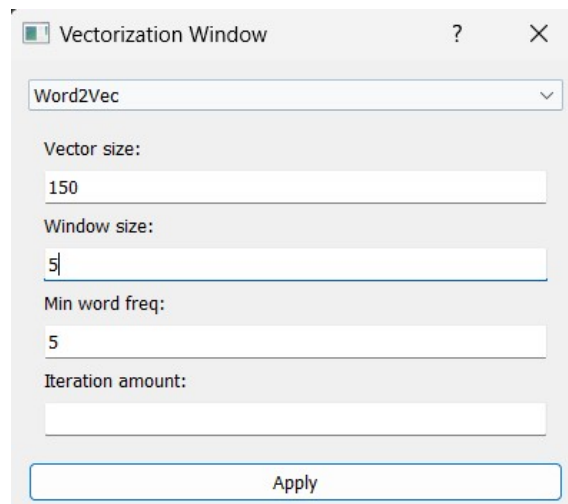
Za pomocą przycisków **Previous** i **Next** można przełączać wykresy odpowiednio do przodu i do tyłu.

3.1.4. Text

Przycisk **Text** otwiera okno dialogowe, w którym użytkownik może określić parametry przetwarzania tekstu. Domyślne okno widać na Rysunku 3.4. Wszystkie parametry są ustawione na wartość true, co oznacza, że są używane.

Użytkownik może wybrać jedną z proponowanych opcji przetwarzania tekstu:

- **alpha** – podstawowa obróbka opisana wcześniej
- **stop-words** – usuwanie stop-words opisane wcześniej
- **lemmatize** – lematyzacja opisana wcześniej



Rysunek 3.5: Przykład okna do konfiguracji wektoryzacji

3.1.5. Wektoryzacja

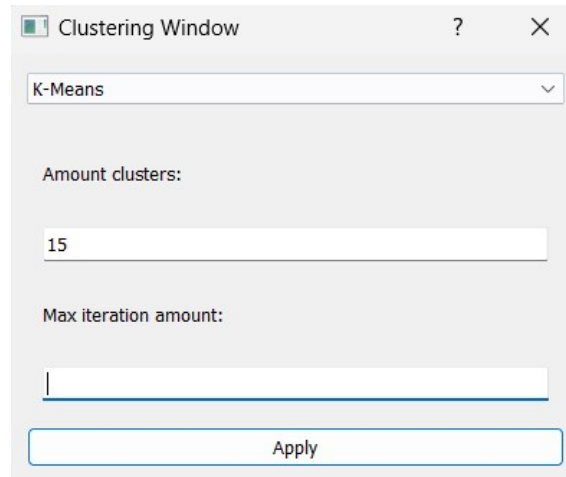
Przycisk Vectorization otwiera okno dialogowe, w którym użytkownik może wybrać jedną z proponowanych metod wektoryzacji i określić parametry dla wybranej metody. Standardowe okno umieszczono na Rysunku 3.5. Początkowo wybrana jest metoda TF-IDF z domyślnymi parametrami. Następnie wymieniono listę dostępnych metod i parametrów dla każdej metody:

- Dostępnymi parametrami do konfiguracji TF-IDF są:
 - **max_df**,
 - **min_df**.
- Dostępnymi parametrami do konfiguracji Word2vec są:
 - **vector_size**,
 - **window**,
 - **min_count**,
 - **sg**.

3.1.6. Cluster

Przycisk Cluster otwiera okno dialogowe, w którym użytkownik może wybrać jedną z proponowanych metod klasteryzacji i określić parametry dla wybranej metody. Domyślne okno widać na Rysunku 3.6. Początkowo wybrana jest metoda K-means z domyślnymi parametrami. Następnie wymieniono listę dostępnych metod i parametrów dla każdej metody:

- Dostępnymi parametrami do konfiguracji K-means są:
 - **n_cluster**,
 - **max_iter**,
 - **n_iter**.
- Dostępnymi parametrami do konfiguracji DBSCAN są:
 - **min_samples**,
 - **eps**.



Rysunek 3.6: Przykład okna do konfiguracji klasteryzacji

3.1.7. Apply

Przycisk Apply otwiera nowe okno z wynikami wykonanej pracy. Okno to zawiera pole z wykresem (2D plot) i pasek narzędziowy (toolbar), gdzie użytkownik może skorzystać z gotowych funkcji zaproponowanych przez bibliotekę Matplotlib do nawigacji po wykresie i innych operacji.

Rozdział 4

Używanie aplikacji

W tym rozdziale opisano proces interakcji z różnymi funkcjami aplikacji i ich wpływ na końcowy wynik pracy na konkretnym przykładzie.

4.1. Przykład tworzenia workspace

Rozpatrując sytuację wyboru filmu z YouTube, do analizy został wybrany film, który jest przedstawiony na Rysunku 4.1.

4.1.1. Pobieranie danych tekstowych

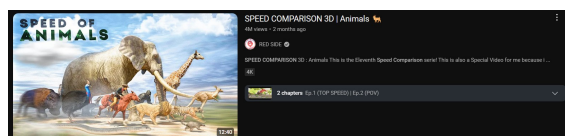
Pierwszym krokiem jest pobranie danych. Po uruchomieniu aplikacji należy otworzyć okno dialogowe YouTube Comments za pomocą paska menu **Comments – YouTube**. Następnie należy wkleić link do filmu i kliknąć przycisk **Find Video**. Po tej operacji okno powinno wyglądać jak na Rysunku 4.2, gdzie wyświetlone są podstawowe informacje o filmie. Po potwierdzeniu, że to właściwy film, należy kliknąć przycisk **Download Comments**. Proces może chwilę potrwać w zależności od liczby komentarzy. Po zakończeniu pomyślnym zostanie wyświetlony komunikat **Comments downloaded successfully**, co oznacza, że komentarze zostały pobrane i zapisane w folderze **Comments** pod nazwą pliku **video_id.txt**, w tym przykładzie jest to **90OC4TQ7uHc.txt**.

4.1.2. Tworzenie Workspace

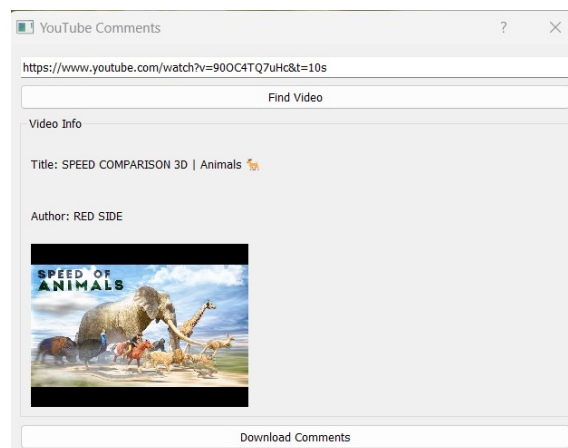
Aby utworzyć projekt (workspace) do pracy z pobranymi komentarzami, należy przejść do menu **File – New Project**, gdzie można wprowadzić ścieżkę do pliku z danymi. Można również skorzystać z przycisku **Browse...** w celu wyboru pliku **90OC4TQ7uHc.txt** i wprowadzenia nazwy Workspace dla identyfikacji. Okno powinno wyglądać jak na Rysunku 4.3.

4.2. Przykład ustawienia parametrów i metod

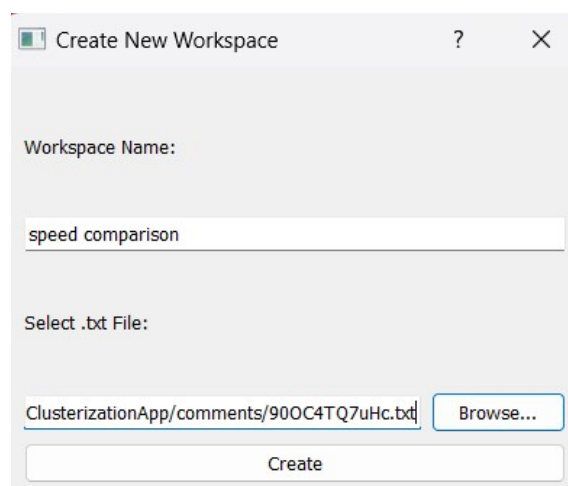
Ten przykład przedstawia różne kombinacje metod i ustawień dla konkretnego przypadku.



Rysunek 4.1: Przykład filmu na YouTube



Rysunek 4.2: Przykład wypełnionego okna pobierania komentarzy



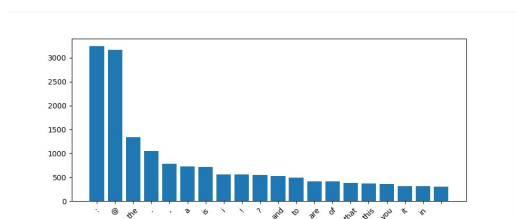
Rysunek 4.3: Przykład wykonanego okna tworzenia nowego obszaru roboczego

4.2.1. Przetwarzanie tekstu

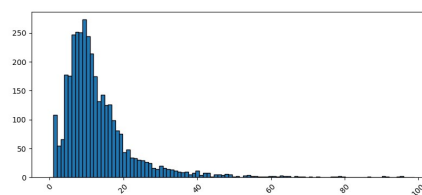
Za pomocą przycisków **Previous** i **Next** można przeglądać zmiany ustawień przetwarzania tekstu oraz zobaczyć wyniki analizy częstotliwości i długości tekstu na wykresach.

Alpha

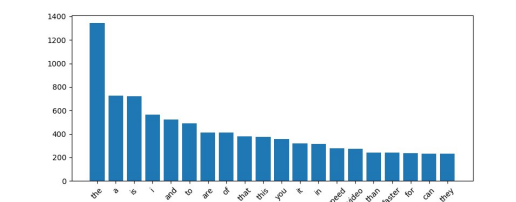
Rozważając zastosowanie parametru **alpha**, można wyraźnie zobaczyć efekt jego działania parametru **alpha** na wykresach **freq_analysis** i **length_analysis**. Na Rysunku 4.4 wśród często występujących tokenów znajduje się duża liczba znaków specjalnych, podczas gdy na Rysunku 4.6 pozostają tylko tokeny bez znaków specjalnych. Rysunki 4.5 i 4.7 pokazują, jak znacząco zmienia się średnia liczba tokenów w komentarzach. To pokazuje, że **alpha** działa poprawnie.



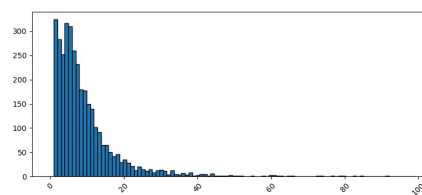
Rysunek 4.4: Przykład analizy częstotliwości tekstu bez TextProcessing



Rysunek 4.5: Przykład analizy długości tekstu bez TextProcessing



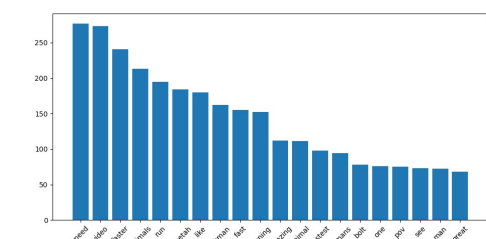
Rysunek 4.6: Przykład analizy częstotliwości tekstu z Alpha



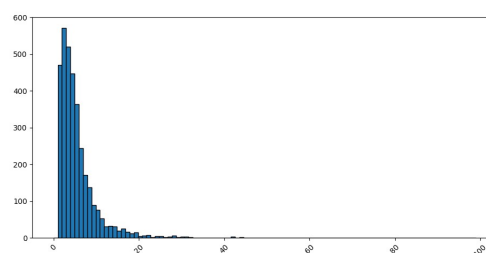
Rysunek 4.7: Przykład analizy długości tekstu z Alpha

Stop-words

Rozważając zastosowanie parametru **stop-words**, można wyraźnie zobaczyć efekt jego działania parametru **stop-words** na wykresach **freq_analysis** i **length_analysis**. Rysunek 4.6 zawiera dużą liczbę przyimków wśród często występujących tokenów, podczas gdy na rysunku 4.8 pozostają tylko rzeczowniki pospolite, które niosą ze sobą najwięcej informacji. Na Rysunki 4.7 i 4.9 pokazują, że średnia liczba tokenów w komentarzu również spada. Wskazuje to, że **stop-words** działają poprawnie.

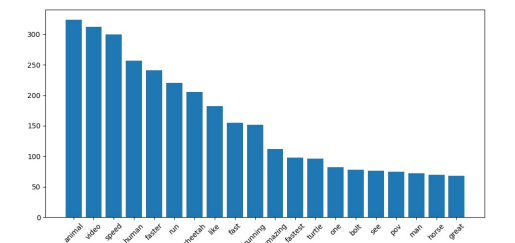


Rysunek 4.8: Przykład analizy częstotliwości tekstu z alpha i stop-words



Rysunek 4.9: Przykład analizy długości tekstu z alpha i stop-words

Rozważając zastosowanie parametru **lemmatize**, efekt jego użycia jest widoczny na wykresie **freq_analysis**. Działanie parametru **lemmatize** najlepiej ilustruje łączenie form zwierząt **animals** i **animal**, co można zaobserwować na Rysunku 4.8 oraz Rysunku 4.10. Po zastosowaniu lematyzacji, **animal** staje się najczęstszą formą. Podobne zmiany można zauważyć w przypadku **humans** i **human**.



Rysunek 4.11: Przykład analizy długości tekstu z alpha i stop-words i lemmatize

Wynik wektoryzacji jest możliwy do oceny dopiero po wizualizacji. W kolejnych sekcjach przedstawione jest wizualne wykorzystanie TF-IDF i Word2Vec z różnymi parametrami.

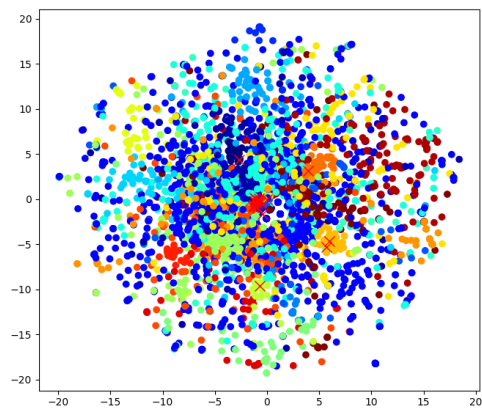
Wykorzystanie metody **TF-IDF** zachodzi na dwa sposoby. Użycie domyślnych parametrów jest przedstawione na Rysunku 4.12 i użycie parametrów **max_df** = 0.5, **min_df** = 0.5 jest przedstawione na Rysunku 4.13. W takich przypadkach analiza skupień jest praktycznie niemożliwa. Dane są bardzo mieszane z najczęściej występującymi tokenami skupionymi w centrum i rzadko występującymi tokenami na krawędziach. Różnica przy zmianie parametrów nie jest bardzo widoczna, ale na Rysunku 4.13 zauważalne są linie kierujące od środka do krawędzi, podczas gdy na Rysunku 4.12 wykres jest równomiernie rozłożony na okręgu.

Zastosowanie metody **Word2Vec** zachodzi na dwa sposoby. Podczas zmiany parametru **vector_size** wykres staje się lekko rozciągnięty. Wykres z domyślnymi parametrami ilustrowany jest na Rysunku 4.14, a z parametrami **vector_size** = 500 jest przedstawiony na Rysunku 4.15. Analiza wydajności klastrowania przy użyciu **Word2vec** staje się bardziej widoczna, a na Rysunku 4.15 z parametrami **vector_size** i Rysunku 4.14 jest widoczny wyraźny podział na różne klastry.

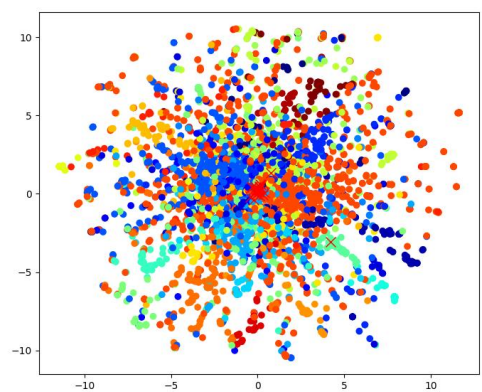
Rozważmy trzy przypadki zastosowania algorytmu **K-means** z liczbą klastrów wynoszącą odpowiednio 5, 15 i 35, przedstawione na Rysunkach 4.16, 4.17 i 4.18.

W pierwszym przypadku wyraźnie widać, że liczba klastrów jest niewystarczająca. Poszczególne grupy nie są prawidłowo odwzorowane, co prowadzi do wewnętrznego podziału w obrębie pojedynczych klastrów.

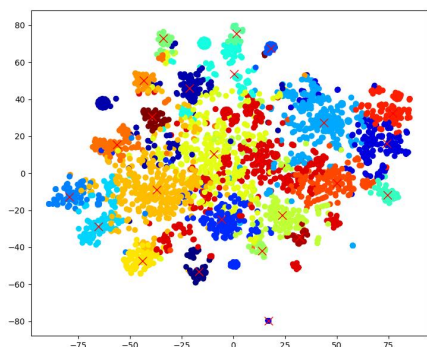
W drugim przypadku również liczba klastrow jest zbyt mała. Pomimo zwiększenia liczby klastrow, nadal występują problemy z ich podziałem, co sugeruje, że ilość klastrow jest niewystarczająca do pełnego odwzorowania struktury danych.



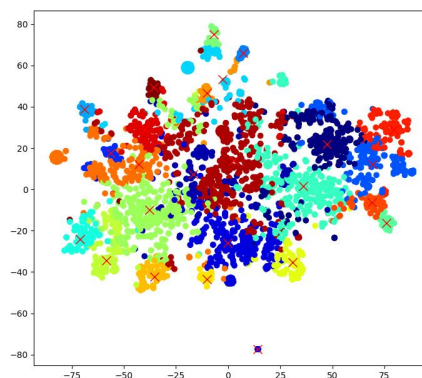
Rysunek 4.12: Przykład TF-IDF przy użyciu parametrów domyślnych



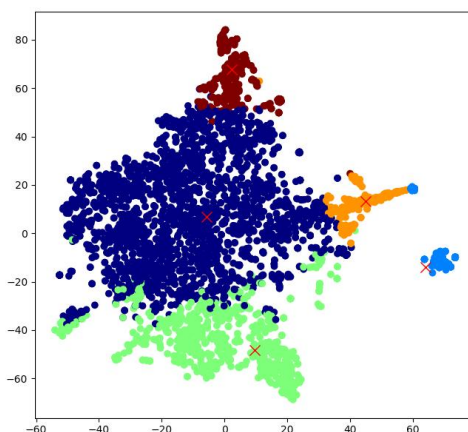
Rysunek 4.13: Przykład TF-IDF przy użyciu niestandardowych parametrów



Rysunek 4.14: Przykład Word2Vec przy `vector_size = 150`



Rysunek 4.15: Przykład Word2Vec przy `vector_size = 500`



Rysunek 4.16: Przykład K-means przy `n_clusters = 5`

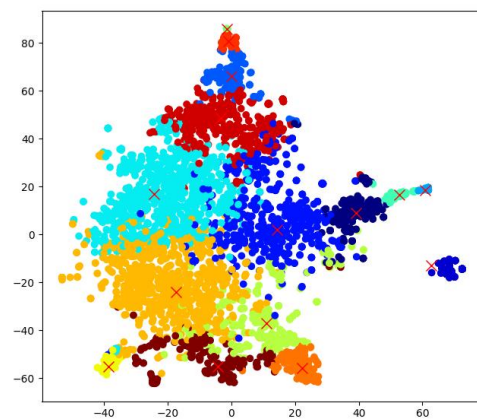
W trzecim przypadku liczba klastrow wynosi 35 i jest już zbliżona do optymalnej. Niemniej jednak, w niektórych miejscach kluczowe regiony są dzielone na mniejsze, zbyteczne klastry, co wskazuje na nieznaczne przekroczenie wymaganej liczby klastrow.

DBSCAN

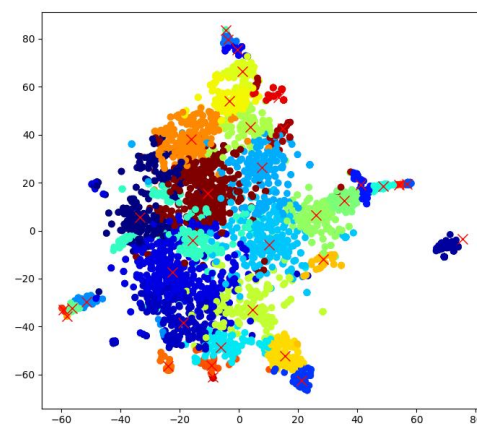
Rozważając cztery przypadki pracy z DBSCAN, warto zwrócić uwagę na zmiany **eps**, przedstawione na Rysunku 4.19 i na Rysunku 4.20 oraz zmiany **min_pts** na Rysunku 4.21 i na Rysunku 4.22. We wszystkich przypadkach zauważalne jest to, że DBSCAN jest bardzo wrażliwy na wybór parametrów. Na Rysunku 4.19 i Rysunku 4.22 większość komentarzy nie jest zawarta w żadnym klastrze, a większość klastrow składa się z bardzo małej liczby komentarzy. Ponieważ parametry były bardzo korzystne dla grupowania, na Rysunku 4.20 i na Rysunku 4.21 większość komentarzy zostało skupionych w jednym dużym klastrze.

4.2.4. Analiza wyników pracy

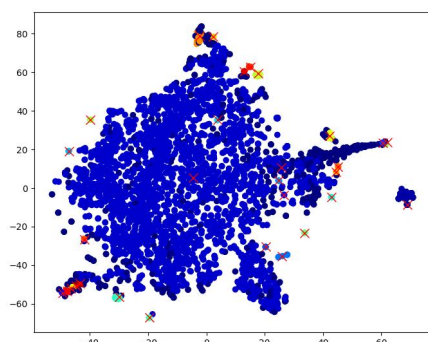
Wyniki analizy danych dla danego przykładu był otrzymany przy użyciu parametrów **Word2vec**, grupowania z parametrami **K-means** i wszystkich metod przetwarzania tekstu.



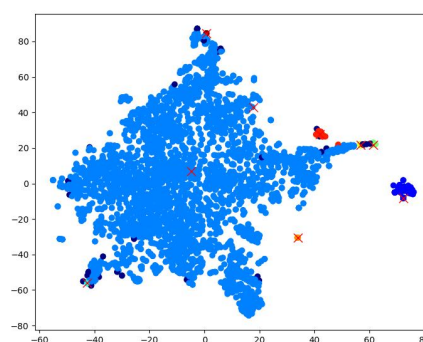
Rysunek 4.17: Przykład K-means przy $n_{\text{clusters}} = 15$



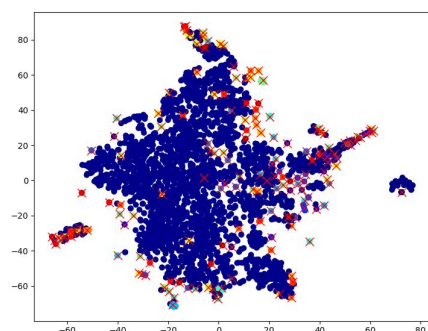
Rysunek 4.18: Przykład K-means przy $n_{\text{clusters}} = 35$



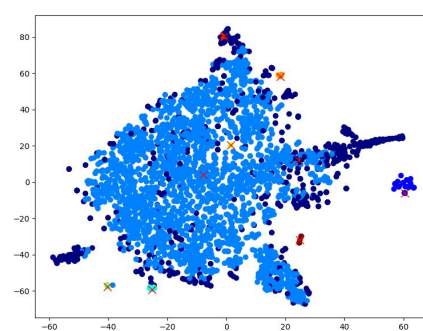
Rysunek 4.19: Przykład DBSCAN przy $\text{eps} = 4$ i $\text{min_pts} = 0.05$



Rysunek 4.20: Przykład DBSCAN przy $\text{eps} = 4$ i $\text{min_pts} = 0.075$

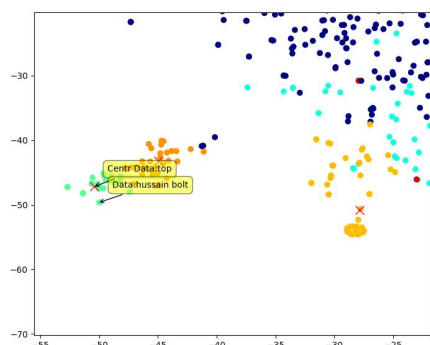


Rysunek 4.21: Przykład DBSCAN przy $\text{eps} = 2$ i $\text{min_pts} = 0.05$

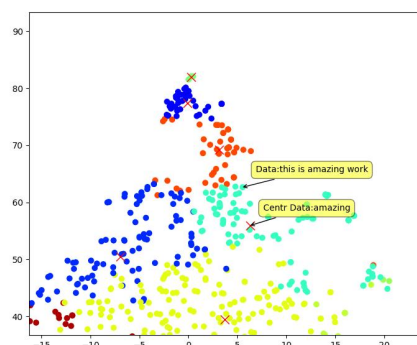


Rysunek 4.22: Przykład DBSCAN przy $\text{eps} = 10$ i $\text{min_pts} = 0.05$

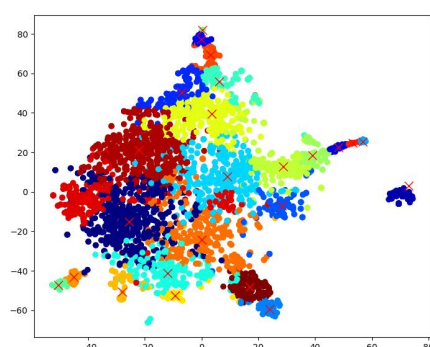
Wynik grupowania pokazano na Rysunku 4.25. Rysunek 4.23 przedstawia grupę komentarzy oddzielonych od części ogólnej w oparciu o informacje o centrum klastra **bolt** i komentarze z tego obszaru. Wydaje się, że komentarze te często wspominają o **Usain Bolt**. Również bardziej podobne komentarze znajdują się bliżej siebie, jak na Rysunku 4.24 **Amazing** i na Rysunku 4.27 **human**. Grupa **Amazing** na Rysunku 4.24 i grupa **Content** na Rysunku 4.26, gdzie często wspomina się o **Video Amazing**, znajdują się blisko siebie.



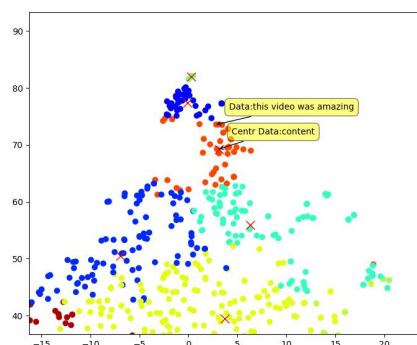
Rysunek 4.23: Przykład częściowego wyniku pracy, przedstawiający klastery o nazwie **Bolt**



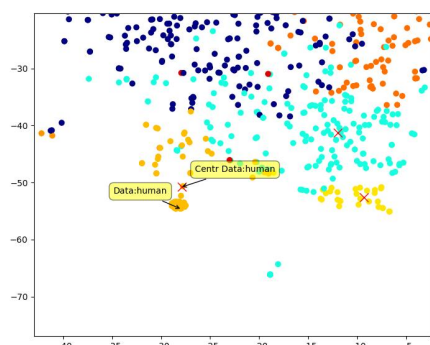
Rysunek 4.24: Przykład częściowego wyniku pracy, przedstawiający klastery o nazwie **Amazing**



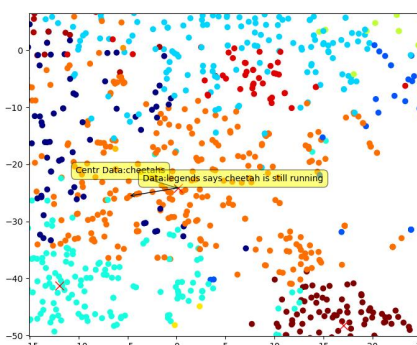
Rysunek 4.25: Przykład całkowitego wyniku pracy



Rysunek 4.26: Przykład częściowego wyniku pracy, przedstawiający klastery o nazwie **Content**



Rysunek 4.27: Przykład częściowego wyniku pracy, przedstawiający klastery o nazwie **Human**



Rysunek 4.28: Przykład częściowego wyniku pracy, przedstawiający klastery o nazwie **Cheetah**

Podsumowanie

Wynikiem pracy jest aplikacja, której funkcją jest implementacja graficznego interfejsu do pobierania danych tekstowych z serwisu YouTube, przedstawienie narzędzi do ustawienia parametrów i wyboru metod do analizy danych tekstowych oraz końcowy wykres odzwierciedlający dane tekstowe.

Aplikacja pozwala na efektywne pobieranie danych tekstowych z komentarzy w serwisie YouTube, co umożliwia badanie opinii i nastrojów użytkowników. Dzięki zastosowaniu interfejsu graficznego użytkownicy mogą łatwo ustawiać parametry analizy oraz wybierać różne metody przetwarzania danych tekstowych, takie jak analiza sentymentu, identyfikacja kluczowych słów czy grupowanie danych.

Podsumowując powyższe wyniki, można podkreślić kilka kluczowych wniosków. Aplikacja jest skutecznym narzędziem do analizy danych tekstowych pochodzących z serwisu YouTube. Pozwala na szybkie pobieranie i przetwarzanie dużej ilości danych, co może być szczególnie użyteczne w badaniach opinii publicznej. Aplikacja oferuje różne metody przetwarzania danych, co pozwala na wszechstronną i obiektywną ocenę danych tekstowych. Wykorzystano metody klasteryzacji, takie jak K-means i DBSCAN oraz metody wektoryzacji, takie jak TF-IDF i Word2Vec. Dodatkowo, zastosowano techniki redukcji wymiarowości, takie jak t-SNE. Użytkownicy mogą wybierać spośród wymienionych algorytmów i technik, co zwiększa elastyczność i dokładność analizy. Graficzny interfejs użytkownika jest intuicyjny i łatwy w obsłudze. Dane na wykresie są przedstawione za pomocą klastrów o różnych kolorach, co ułatwia szybkie zorientowanie się wśród różnych klastrów na wykresie. Kolorowe klastry pozwalają na łatwe rozróżnienie i interpretację wyników analizy.

Pracę można rozwinąć poprzez integrację z innymi platformami społecznościowymi, takimi jak Twitter czy Facebook oraz wdrożenie bardziej zaawansowanych algorytmów analizy tekstu, takich jak modele głębokiego uczenia. Możliwe jest także dodanie funkcji automatycznego doboru optymalnych parametrów analizy danych.

Bibliografia

- [1] Impossibility Theorem for Clustering Theorem for Clustering, <https://www.cs.cornell.edu/home/kleinber/nips15.pdf>
- [2] A. K. Jain, R. C. Dubes, *Algorithms for Clustering Data*, Prentice-Hall, Inc., 1988.
- [3] Daniel Jurafsky, James H. Martin, *Speech and Language Processing*, <https://web.stanford.edu/~jurafsky/slp3/ed3book.pdf>, Stanford University., 2024.
- [4] C. C. Aggarwal, C. Zhai (Eds.), *Mining Text Data*, Springer, 2012.
- [5] A. Y. Ng, M. I. Jordan, Y. Weiss, *On Spectral Clustering: Analysis and an Algorithm*, Advances in Neural Information Processing Systems, 14, 2001.
- [6] Clustering in Information Retrieval, <https://nlp.stanford.edu/IR-book/html/htmledition/clustering-in-information-retrieval-1.html>
- [7] Trip Database Blog, *Clustering Search Results Using Carrot2*, <https://blog.tripdatabase.com/2024/04/28/clustering-search-results-using-carrot2/>
- [8] YouTube Data API *YouTube Data API*, <https://developers.google.com/youtube/v3/docs?hl=pl>
- [9] Steven Bird, Ewan Klein, Edward Loper, *Natural Language Processing with Python*, O'Reilly Media, 2009.
- [10] F. Pedregosa et al., *Scikit-learn: Machine Learning in Python*, Journal of Machine Learning Research, 12, pp. 2825-2830, 2011.
- [11] Radim Řehůřek, Petr Sojka, *Software Framework for Topic Modelling with Large Corpora*, In Proceedings of the LREC 2010 Workshop on New Challenges for NLP Frameworks, pp. 45-50, 2010.
- [12] G. Salton, A. Wong, C. S. Yang, *A Vector Space Model for Automatic Indexing*, Communications of the ACM, 18(11), pp. 613-620, 1975.
- [13] Tomas Mikolov, Kai Chen, Greg Corrado, Jeffrey Dean, *Efficient Estimation of Word Representations in Vector Space*, arXiv preprint arXiv:1301.3781, 2013.
- [14] J. MacQueen, *Some Methods for Classification and Analysis of Multivariate Observations*, In Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability (Vol. 1, No. 14, pp. 281-297), 1967.
- [15] M. Ester, H. P. Kriegel, J. Sander, X. Xu, *A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise*, In KDD (Vol. 96, No. 34, pp. 226-231), 1996.
- [16] L. van der Maaten, G. Hinton, *Visualizing Data using t-SNE*, Journal of Machine Learning Research, 9(Nov), pp. 2579-2605, 2008.
- [17] Mark Summerfield, *Rapid GUI Programming with Python and Qt*, Prentice Hall, 2007.
- [18] John D. Hunter, *Matplotlib: A 2D Graphics Environment*, Computing in Science And Engineering, 9(3), pp. 90-95, 2007.

Spis listingów

2.1. Funkcja usuwająca niealfabetyczne symbole	12
2.2. Funkcja usuwająca stop-words	12
2.3. Funkcja lematyzująca tokeny	13
2.4. Funkcja przeprowadzająca analizę częstotliwości tokenów	13
2.5. Funkcja przeprowadzająca analizę długości tokenów	13
2.6. Funkcja wektoryzująca korpus za pomocą TF-IDF	15
2.7. Funkcja wektoryzująca za pomocą Word2Vec	16
2.8. Pseudokod algorytmu K-means	17
2.9. Przykład użycia algorytmu K-Means	18
2.10. Pseudokod algorytmu DBSCAN	19
2.11. Przykład użycia algorytmu DBSCAN	19
2.12. Przykład użycia t-SNE do osadzania danych	20

Spis rysunków

2.1. Interfejs graficzny po wprowadzeniu linku do filmu (krok 1)	11
2.2. Interfejs graficzny po uzyskaniu informacji o filmie (krok 3)	11
3.1. Przykład początkowego okna	22
3.2. Przykład wypełnionego okna	22
3.3. Przykład początkowego okna do pobierania komentarzy	22
3.4. Przykład okna do konfiguracji przetwarzania tekstu	23
3.5. Przykład okna do konfiguracji wektoryzacji	24
3.6. Przykład okna do konfiguracji klasteryzacji	25
4.1. Przykład filmu na YouTube	27
4.2. Przykład wypełnionego okna pobierania komentarzy	28
4.3. Przykład wykonanego okna tworzenia nowego obszaru roboczego	28
4.4. Przykład analizy częstotliwości tekstu bez TextProcessing	29
4.5. Przykład analizy długości tekstu bez TextProcessing	29
4.6. Przykład analizy częstotliwości tekstu z Alpha	29
4.7. Przykład analizy długości tekstu z Alpha	29
4.8. Przykład analizy częstotliwości tekstu z alpha i stop-words	29
4.9. Przykład analizy długości tekstu z alpha i stop-words	29
4.10. Przykład analizy częstotliwości tekstu z alpha i stop-words i lemmatize	30
4.11. Przykład analizy długości tekstu z alpha i stop-words i lemmatize	30
4.12. Przykład TF-IDF przy użyciu parametrów domyślnych	31
4.13. Przykład TF-IDF przy użyciu niestandardowych parametrów	31
4.14. Przykład Word2Vec przy vector_size = 150	32
4.15. Przykład Word2Vec przy vector_size = 500	32
4.16. Przykład K-means przy n_clusters = 5	32
4.17. Przykład K-means przy n_clusters = 15	33
4.18. Przykład K-means przy n_clusters = 35	33
4.19. Przykład DBSCAN przy eps = 4 i min_pts = 0.05	34
4.20. Przykład DBSCAN przy eps = 4 i min_pts = 0.075	34
4.21. Przykład DBSCAN przy eps = 2 i min_pts = 0.05	34
4.22. Przykład DBSCAN przy eps = 10 i min_pts = 0.05	34
4.23. Przykład częściowego wyniku pracy, przedstawiający klaster o nazwie Bolt	35
4.24. Przykład częściowego wyniku pracy, przedstawiający klaster o nazwie Amazing .	35
4.25. Przykład całkowitego wyniku pracy	35
4.26. Przykład częściowego wyniku pracy, przedstawiający klaster o nazwie Content .	35
4.27. Przykład częściowego wyniku pracy, przedstawiający klaster o nazwie Human . .	35
4.28. Przykład częściowego wyniku pracy, przedstawiający klaster o nazwie Cheetah .	35